4. Functions and Scope

1. What is a function in C++? Explain the concept of function declaration, definition, and calling.

1. Function Declaration

- Tells the compiler about function name, return type, and parameters

```
int add(int, int);
```

2. Function Definition

- Actual body where logic is written

```
int add(int a, int b) {
   return a + b;
}
```

3. Function Calling

- Calling the function to execute

```
int result = add(5, 3);
```

2. What is the scope of variables in C++? Differentiate between local and global scope.

Scope means where a variable can be accessed in a program.

Local Scope

- Declared inside a function or block
- Accessible only inside that function

```
void fun() {
   int x = 10;  // local variable
}
```

Global Scope

- Declared outside all functions
- Accessible throughout the program

```
int x = 20;  // global variable

void fun() {
   cout << x;
}
```

Difference

- Local → limited access
- Global → accessible everywhere

3. Explain recursion in C++ with an example.

```
int fact(int n) {
   if(n == 0)
      return 1;   // base case
   else
      return n * fact(n - 1);
}
```

Call:

```
fact(5);  // Output: 120
```

4. What are function prototypes in C++? Why are they used?

```
int add(int, int);  // prototype

int main() {
```

```
    cout << add(2, 3);
}

int add(int a, int b) {
    return a + b;
}
```

Why they are used

- Inform compiler about function before use
- Avoid errors
- Allow functions to be defined after main()
- Improve code organization

5. Arrays and Strings

1. What are arrays in C++? Explain the difference between single-dimensional and multi- dimensional arrays.

Single-Dimensional Array (1D)

- Stores elements in a single row

```
int arr[5] = {1, 2, 3, 4, 5};
```

👉 Access using index:

```
cout << arr[0];  // 1
```

Multi-Dimensional Array (2D)

- Stores data in rows and columns (table form)

```
int arr[2][3] = {
    {1, 2, 3},
```

```
    {4, 5, 6}
};
```

👉 Access:

```
cout << arr[0][1];  // 2
```

2. Explain string handling in C++ with examples.

1. Character Array

```
char str[] = "Hello";
```

2. string class (Recommended)

```
#include<iostream>
#include<string>
using namespace std;

int main() {
    string str = "Hello";
    cout << str;
}
```

Input String

```
string name;
cin >> name;        // single word
getline(cin, name); // full line
```

3. How are arrays initialized in C++? Provide examples of both 1D and 2D arrays.

1D Array Initialization

```
int a[5] = {1, 2, 3, 4, 5};
```

OR

```
int a[] = {1, 2, 3};
```

2D Array Initialization

```
int b[2][2] = {
   {1, 2},
   {3, 4}
};
```

OR

```
int b[2][2] = {{1,2},{3,4}};
```

4. Explain string operations and functions in C++.

1. Length

```
string s = "Hello";
cout << s.length();
```

2. Concatenation

```
string a = "Hi";
string b = "There";
cout << a + b;
```

3. Compare

```
if(a == b)
```

4. Access Character

```
cout << s[0];
```

5. Substring

```
cout << s.substr(0, 3);
```

6. Append

```
s.append(" World");
```

6. Introduction to Object-Oriented Programming

1. Explain the key concepts of Object-Oriented Programming (OOP).

1. Encapsulation

- Wrapping data and functions together in a class
- Data hiding for security

2. Inheritance

- One class can use properties of another class
- Promotes code reuse

3. Polymorphism

- Same function behaves differently
- Example: function overloading

4. Abstraction

- Hides complex details and shows only essential features

2. What are classes and objects in C++? Provide an example.

Class

- Blueprint or template to create objects

Object

- Instance of a class

Example:

```
#include<iostream>
using namespace std;

class Student {
public:
   int id;
   string name;
};

int main() {
   Student s1;   // object
   s1.id = 1;
   s1.name = "Rahul";

   cout << s1.id << " " << s1.name;
   return 0;
}
```

3. What is inheritance in C++? Explain with an example.

```
#include<iostream>
using namespace std;

class Parent {
public:
   void show() {
      cout << "Parent class";
   }
};

class Child : public Parent {
};

int main() {
   Child c;
   c.show();  // inherited function
   return 0;
}
```

4. What is encapsulation in C++? How is it achieved in classes?

Achieved using:

- Classes
- Access specifiers (private, public, protected)

Example:

```
#include<iostream>
using namespace std;

class Student {
private:
```

```
    int marks;

public:
    void setMarks(int m) {
        marks = m;
    }

    int getMarks() {
        return marks;
    }
};

int main() {
    Student s;
    s.setMarks(90);
    cout << s.getMarks();
    return 0;
}
```